Cleanup and exception propagation

CSC103 Programming Fundamentals / Lecture 25

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Trace a try body that prints A and throws, a matching catch that prints B, and a finally block that prints C. What if the catch rethrows?

Place the failing operation inside an inner try.

Learning objectives

- Trace nested handlers and finally
- Rethrow or translate an exception
- Preserve the cause of a failure

CLO-3
Propagation across calls
The finally clause
Rethrowing an exception
Chained exceptions

Propagation across calls

- An unhandled exception leaves the current method.
- The runtime continues searching in active callers.
- Stack traces report the chain of calls involved in the failure.

Example

```
main calls loadCount
loadCount calls parseInt
parseInt throws NumberFormatException
A matching caller catch handles it
```

The finally clause

- finally normally runs when control leaves try-catch.
- It can run after a return or an exception.
- It is not guaranteed after forced process termination or a JVM crash.

Example

```
try {
  System.out.println("work");
} finally {
  System.out.println("cleanup");
}
```

Rethrowing an exception

- A handler may record useful context and rethrow the same exception.
- Rethrowing lets a higher layer decide how to recover.
- Avoid duplicate reports at every layer.

Example

```
catch (NumberFormatException ex) {
  throw ex;
}
```

Chained exceptions

- Translation can express failure in the caller domain.
- Pass the original exception as the cause.
- Preserved causes help explain the underlying problem.

Example

```
catch (NumberFormatException ex) {
  throw new IllegalArgumentException(
      "Invalid record count", ex);
}
```

A handler can pass responsibility upward

- A catch may perform a local action and then rethrow the same exception.
- Its finally block still runs while control leaves the protected structure.
- An outer handler can then decide how the whole operation should respond.

Example

```
Inner work prints A.
Inner catch prints B and rethrows.
Inner finally prints C.
Outer catch prints D.
```

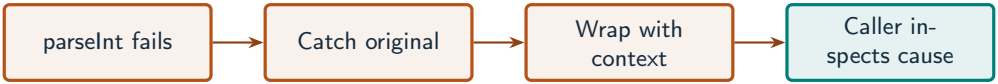
A cause preserves diagnostic history

- A translated exception can state which application operation failed.
- Its cause identifies the lower-level failure that explains why.
- Replacing an exception with a message alone discards that useful chain.

Example

```
throw new IllegalArgumentException(
    "Invalid mark field", originalException);
```

Preserve the cause when adding context



What to notice

A higher-level message explains the failed operation while the cause retains the original failure.

Key distinctions

Action in catch	Original failure retained?	Where control goes next
Handle and finish	Available in local catch	After try-catch-finally
throw ex	Same exception	Outer compatible handler
Throw wrapper with cause	As a cause	Outer compatible handler
Return from finally	Can suppress failure	Caller receives the return

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static int readCount(String text) {
  try { return Integer.parseInt(text); }
  catch (NumberFormatException ex) {
    throw new IllegalArgumentException("Invalid record count", ex);
  }
}
```

First example: execution

```
try {
  readCount("bad");
} catch (IllegalArgumentException ex) {
  System.out.println(ex.getMessage());
  System.out.println(ex.getCause().getClass().getSimpleName());
} finally {
  System.out.println("Cleanup");
}
```

First example: result

Invalid record count
NumberFormatException
Cleanup
The wrapper retains the original parse failure as its cause.

Rethrowing an exception
Chained exceptions

Guided practice

Trace a try body that prints A and throws, a matching catch that prints B, and a finally block that prints C. What if the catch rethrows?

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Place the failing operation inside an inner try.
- Print B in its catch, rethrow, and print C in its finally.
- Use an outer catch to show that propagation continues after cleanup.

The next program uses fixed inputs so its result can be reproduced.

Complete solution (1/2)

```
public class Solution25 {
  public static void main(String[] args) throws Exception {
    try {
      try {
        System.out.println("A");
        Integer.parseInt("bad");
      } catch (NumberFormatException ex) {
        System.out.println("B");
        throw ex;
```

Complete solution (2/2)

```
      } finally {
        System.out.println("C");
      }
    } catch (NumberFormatException ex) {
      System.out.println("D");
    }
  }
}
```

Execution trace

Order	Location	Action	Output
1	Inner try	Start then parse failure	A
2	Inner catch	Report then rethrow	B
3	Inner finally	Cleanup path	C
4	Outer catch	Handle propagated failure	D

Follow the changing state in execution order.

Solution result

A
B
C
D

Why this result follows
Use an outer catch to show that propagation continues after cleanup.

A common incorrect approach

finally { return 0; }
// The try body may throw an important exception.

Example

A `return` from `finally` suppresses the pending exception. Remove that `return` and keep cleanup separate from the method result.

Boundary and failure checks

Check the stated input assumptions.
Wrap a numeric parsing failure with the input-field name and original cause. Show both messages.

Matching handler: A, B, C, then later code may continue. Rethrow: A, B, C, then propagation continues to a caller.

Check your understanding

What information does a cause preserve? Does finally guarantee recovery?

Write a prediction and explain the rule that supports it.

Answer and explanation

The original failure behind a translated exception. No, finally performs cleanup but a failure may still propagate.

A return from finally suppresses the pending exception. Remove that return and keep cleanup separate from the method result.

Compare cases and predict outcomes

Exception part	Example	Purpose
Wrapper type	IllegalArgumentException	Caller-facing contract
Wrapper message	Invalid record count	Operation context
Cause	NumberFormatException	Underlying failure

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Wrap a parsing failure as IllegalArgumentException with message "Invalid count". Preserve the original exception as its cause.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
static int parseCount(String text) {
  try {
    return Integer.parseInt(text);
  } catch (NumberFormatException ex) {
    throw new IllegalArgumentException("Invalid count", ex);
  }
}
```

- The two-argument constructor retains ex as the cause.
- getMessage() gives the added context; getCause() reaches the original exception.
- A finally block should avoid returning, because that can suppress a pending result or failure.

Lecture summary

- nested handlers and finally
- Rethrow or translate an exception
- Preserve the cause of a failure

- Place the failing operation inside an inner try.
- Print B in its catch, rethrow, and print C in its finally.
- Use an outer catch to show that propagation continues after cleanup.

After-class practice

Wrap a numeric parsing failure with the input-field name and original cause. Show both messages.

Syllabus reading
Deitel Ch 11
Next lecture: Programming environments and debugging